

Debugging: Uma revisão sistemática sobre a integração de IA generativa e gamificação no ensino de programação

1st Thuany Guimarães Pereira

Centro de Informática - CIn

Universidade Federal de

Pernambuco

Recife, Brazil

0009-0007-4822-3800

2nd Douglas Araújo Silva

Centro de Informática - CIn

Universidade Federal de

Pernambuco

Recife, Brazil

0009-0006-1448-3164

3rd Nathanael Noberto da Silva

Centro de Informática - CIn

Universidade Federal de

Pernambuco

Recife, Brazil

0000-0002-8153-0569

4st André Luiz Bartholomeu Ribeiro

Centro de Informática - CIn

Universidade Federal de Pernambuco

Recife, Brazil

0009-0002-7809-1160

5nd Luiz Guyllherme Avelino Rodrigues

Centro de Informática - CIn

Universidade Federal de Pernambuco

Recife, Brazil

0009-0002-5328-6949

Abstract— *Este trabalho apresenta uma Revisão Sistemática de Literatura (RSL) focada no mapeamento do estado da arte do ensino e suporte ao debugging em turmas iniciantes de programação de software (CS1). A pesquisa investiga a mediação por meio de gamificação e Inteligência Artificial Generativa (IAG). Foram analisados 10 estudos primários com alto rigor metodológico, revelando uma ruptura metodológica onde LLMs atuam como tradutores semânticos de erros. Os resultados apontam um paradoxo: enquanto a IAG reduz drasticamente a frustração e a carga cognitiva, ela gera riscos de "ilusão de competência" e dependência algorítmica. Conclui-se que a eficácia dessas tecnologias depende de um delineamento instrucional socrático e gamificado.*

Keywords— *Debugging, Gamificação, IA Generativa, Programadores Iniciantes, RSL.*

I. INTRODUÇÃO

Ao pensar sobre as nuances da educação em computação contemporânea, a importância da construção de uma base consistente do conhecimento dos estudantes iniciantes da área vem como uma reflexão necessária e imediata para garantir o nível de qualidade durante as etapas posteriores do processo de aprendizagem e futura atuação profissional dos estudantes. O desenvolvimento dessas competências, especialmente nos que tange aos cursos de introdução à programação, exige do estudante o domínio de conceitos complexos, que por sua vez, demandam de feedbacks formativos adequados para manter a motivação e retenção dos estudantes em seus respectivos cursos, conforme [1].

Seguindo as reflexões de [2], percebemos que quando imerso ao momento prático do processo de *debugging*, é esperado que o estudante iniciante de programação se depare com mensagens de erro, e no caso da não existência de um suporte pedagógico capaz de o fazer entender o significado dessa mensagem, essa prática torna-se uma experiência frustrante e que causa uma sobrecarga cognitiva a esse estudante. É nesse ponto que podemos perceber que a falta de suporte para compreensão dos erros induz os estudantes ao ciclo de tentativa e erro apenas para satisfazer o compilador, sem a compreensão da causa daquela mensagem.

Educadores diariamente vêm buscando abordagens inovadoras com o foco de mitigar a frustração e a sobrecarga cognitiva do *debugging* em CS1. Percebe-se que a literatura tem mapeado o uso de jogos educacionais para apoiar o ensino e a aprendizagem de programação, assim como o estado da arte sobre gamificação em cursos de STEM, mostrando um impacto extremamente positivo no engajamento, motivação e atitude de alunos de programação [3]. A gamificação tem se mostrado como uma abordagem interessante para esse contexto, pois a aplicação de seus elementos no ensino de programação aumenta a motivação e melhora a compreensão dos conteúdos, conforme [4], onde na prática, a motivação tem um aumento refletido pela compreensão de testes de software. O uso de jogos sérios é uma das formas práticas de aplicar a gamificação, pois consistem no emprego de tecnologias e ambientes de jogos digitais aplicados a um contexto educacional, promovendo um aprendizado centrado no aluno a partir de interatividade e feedback

imediatamente, tendo sua natureza imersiva como impulso para que o estudante queira investigar e experimentar os erros [5]. Estudos empíricos demonstram que a utilização desses jogos reduz a intenção de evasão, além de melhorar de forma significativa a autopercepção e a confiança desses aprendizes ao aplicar habilidades de debugging [5].

Apesar dos perceptíveis benefícios da gamificação na resiliência do aprendiz a longo prazo, os obstáculos metacognitivos para a resolução de problemas de programação limitam o avanço dos programadores iniciantes. Por não perceberem a raiz dessas dificuldades, os estudantes consideram a utilização de Inteligência Artificial Generativa (IAG), como as ferramentas de Grande Modelo de Linguagem (LLM), como um atalho para obter suporte para a resolução dos erros e contornar as situações, principalmente pela disponibilidade imediata de feedback [6]. Contudo, seu uso para construção integral de códigos para solucionar as demandas traz malefícios cognitivos e déficits nas próximas etapas de aprendizagem e subsequente atuação profissional, se fazendo necessário considerar que essa forma de suporte da IAG, conforme estudos como [7] indicam a ampliação dessas dificuldades cognitivas, trazendo uma perigosa “ilusão de competência” no iniciante. Por fim, a IAG falha em oferecer um suporte voltado ao pensamento de eliminação da necessidade de conhecimentos sobre as múltiplas etapas do processo de debugging para mitigar os problemas do código, uma vez que exige que o iniciante tenha essa habilidade para que consiga compreender e consertar o código que foi gerado pela própria máquina, conforme demonstrado por [8]. Sem o devido conhecimento, o código gerado pela máquina transfere a carga cognitiva, ao invés de eliminá-la.

Embora existam trabalhos voltados para a utilização de forma isolada da gamificação e da IAG no ensino de programação, nota-se uma certa falta de mapeamento sobre como a combinação e estruturação delas pode ser realizada com foco em promoção da aprendizagem concreta das habilidades necessárias para o processo de *debugging* para iniciantes, mas de uma forma saudável, sem a geração de dependência tecnológica, capacitando de forma global a independência do aprendiz.

Com esse trabalho, pretendemos mapear o estado da arte do ensino e suporte ao debugging em turmas iniciantes de programação de software, utilizando a mediação por meio de gamificação e IAG, realizando uma Revisão Sistemática de Literatura (RSL) para identificar

as tendências, desafios metodológicos e direcionar trabalhos futuros para a área de Informática na Educação.

II. TRABALHOS RELACIONADOS E FUNDAMENTAÇÃO TEÓRICA: O CENÁRIO ATUAL DO ENSINO DE DEBUGGING

Esta seção apresenta o panorama atual da literatura no que tange aos desafios do ensino de depuração de código (*debugging*) e as abordagens tecnológicas utilizadas para mitigá-los. Nesse caminho, a Subseção II.A fundamenta as barreiras cognitivas e emocionais enfrentadas por programadores iniciantes. Já a Subseção II.B discute os impactos da busca por ajuda não supervisionada em ferramentas de IAG. Em seguida, a Subseção II.C aborda como o cenário de dicas socráticas e ambientes gamificados tem sido relatado pela literatura, finalizando em II.D onde é explicitada a lacuna científica relacionada à integração híbrida dessas tecnologias.

A. O labirinto emocional e metacognitivo do debugging

O ensino de debugging para estudantes iniciantes é um dos maiores desafios enfrentados pedagogicamente na computação. A literatura revela que por falta de consciência metacognitiva, o aprendizado de programação torna-se um caminho mais difícil, pois esse déficit prejudica a elaboração de estratégias metacognitivas para a compreensão e execução das atividades, sendo a metacognição uma habilidade crucial no aprendizado de programação [7]. Ao se deparar com uma mensagem de erro, sem a devida consciência metacognitiva para analisar logicamente o erro, os estudantes relatam que “entram em pânico” e, na tentativa de resolver o problema, iniciam uma saga de tentativa e erro cega em busca de se livrar do erro apresentado na mensagem, para corrigi-lo mesmo sem uma compreensão exata do que se trata [9].

Esse “pânico”, gera consequências no processo de aprendizagem, pois o choque da discrepância entre o que o aluno esperava que o código fizesse e o erro real apresentado, acaba gerando um “impasse cognitivo” (Confusão) [10]. Com a dificuldade na elaboração de estratégias metacognitivas para analisar logicamente a situação, os aprendizes não sabem como sair desse impasse, o que acaba por transformar a confusão em uma frustração severa (estresse lógico), o que gera a sensação de incapacidade, destruindo a confiança desse aprendiz, sendo essa uma das maiores causas de evasão de seus respectivos cursos em sua fase inicial [11]

B. A busca por Ajuda e a “Ilusão de Competência” gerada pela IAG

Como alternativa para fuga do estresse e frustração incapacitante, o comportamento do aprendiz tende frequentemente a realizar a busca por ajuda executiva e soluções com atalhos rápidos, ao invés de aprofundar seus conhecimentos e encontrar soluções profundas e de maior impacto no seu aprendizado [12]. É nesse contexto que as ferramentas de IAG tornaram-se um socorro imediato para os estudantes, mostrando-se útil pela rapidez do retorno. O problema, contudo, reside na forma de utilização: por falta de instrução, os aprendizes terceirizam o debugging para a máquina, sem tentar previamente resolver independentemente ou validar se as respostas da IAG estão adequadas para resolver o problema [12] [13].

A literatura recente vem alertando sobre perigos inerentes dessa prática, onde relatam que o uso livre da IAG agrava a resolução dos impasses cognitivos causados pelo processo de debugging, ao invés de funcionar como esse socorro imediato de forma, além de eficaz, saudável. A entrega de um código pronto no lugar da devida explicação sobre a construção e funcionamento, promove uma dissonância cognitiva no estudante, em que ele passa a ter a perigosa “ilusão de competência”, inviabilizando o desenvolvimento do raciocínio lógico autônomo intrínseco nesse tipo de atividade [7].

C. Estratégias de Mediação Atuais: Dicas Socráticas e Gamificação

Uma forma de combate da terceirização do raciocínio para a máquina é a mudança na forma do estudante interagir com as ferramentas de IAG. Conforme pesquisas recentes, para essa mudança de comportamento é necessário que o design das ferramentas educacionais “dome” os LLMs para que eles atuem apenas como tutores, fornecendo feedback contextualizado e dicas estruturadas, o que automaticamente induz o aluno a manter o foco conceitual no que é exigido pelo processo de debugging e a consequente resolução dos erros com menos tentativas falhas [14] [15].

Ao mesmo tempo em que as inovações instrucionais com utilização de IAG são recomendadas para a resolução desse problema, os trabalhos que falam sobre a gamificação vêm reconhecendo e validando esse mecanismo como promissor no ensino de programação, mostrando-se eficaz na barreira emocional do debugging, servindo para melhorar a participação do estudante e impactar positivamente no aprendizado [3], ao mesmo

tempo em que a natureza imersiva e envolvente dos jogos incentiva os alunos a níveis mais elevados de investigação, experimentação e reutilização [5], assim como promove o uso de scaffolding e testes para apoiar o ensino de programação [4] antes mesmo da popularização da IAG.

D. A Lacuna da Integração Híbrida no Ensino de Programação

Levando em consideração a possibilidade da geração de dependência ao uso de IAG, é preciso sinalizar que essa tecnologia também tem a capacidade de oferecer dicas vitais. Pesquisadores vêm defendendo uma profunda mudança pedagógica para alinhar o seu uso de forma psicologicamente saudável, com a criação de métodos integradores que possam ativamente utilizar a IAG em ambientes gamificados (Jogos Sérios) com foco na garantia de desenvolvimento de conhecimento crítico a partir de atividades práticas, ou seja, ensinar o aprendiz a pensar enquanto realiza as atividades, deixando de lado a guerra ferrenha pela detecção do uso de IAG, que seria uma perda de tempo [16] [17]. Pensando nisso, vemos que ainda é deficiente a literatura quando se fala na junção do ensino de debugging para alunos iniciantes com a integração híbrida entre gamificação e IAG, que tem se mostrado interessante para contornar os problemas existentes, como abordado em [17].

Pensando em preencher essa lacuna na literatura, o presente artigo propõe mapear como o está sendo desenhado e estruturado no estado da arte as abordagens interativas conjuntas, buscando entender os impactos na autonomia e engajamento de estudantes iniciantes em programação.

III. Metodologia da Pesquisa

Para investigar a lacuna identificada na seção anterior, este trabalho conduziu uma Revisão Sistemática de Literatura (RSL) seguindo as diretrizes de Kitchenham [18]. Esse método foi escolhido para garantir o rigor científico na busca, seleção e extração dos dados. A revisão tem como objetivo principal o mapeamento do estado da arte recente sobre o ensino de depuração de código (*debugging*) para alunos iniciantes, com foco na utilização de metodologias ativas, especificamente na integração entre a gamificação e ferramentas de IAG. Dessa forma, busca-se estruturar as dificuldades encontradas, as inovações que vêm sendo utilizadas e fornecer uma base teórica e prática para o design

instrucional com método para futuras intervenções educacionais.

A. *Questões de pesquisa*

Como meio para alcançar o objetivo proposto e guiar o processo de revisão, o estudo foi embasado pela seguinte Questão Central de Pesquisa (QC):

- QC: Quais abordagens baseadas em gamificação e Inteligência Artificial Generativa têm sido utilizadas de forma combinada ou estruturada para apoiar o processo de depuração de software e promover a autonomia de programadores iniciantes?

Para desdobrar a questão central e direcionar a extração de dados, foram definidas quatro Questões Secundárias (QS):

- QS1: Quais são as principais abordagens utilizadas para ensinar o processo de depuração a programadores iniciantes, incluindo o uso de gamificação e Inteligência Artificial Generativa?
- QS2: Quais benefícios e resultados têm sido observados no uso de gamificação e Inteligência Artificial Generativa no ensino de depuração (ex.: engajamento, aprendizagem, autonomia)?
- QS3: Quais desafios, limitações e dificuldades são relatados na aplicação de gamificação e Inteligência Artificial Generativa no processo de ensino de depuração?
- QS4: Qual é o impacto dessas abordagens (gamificação e IA generativa) no desenvolvimento do raciocínio lógico, na autonomia e na redução da frustração de estudantes iniciantes?

Para responder essas questões, formularam-se elementos para um protocolo de revisão detalhando a estratégia de busca, estruturada pelo método PICOC, e os critérios de seleção, que serão detalhados nas subseções a seguir.

B. *Estratégia de busca e PICO*

Para definir os termos estruturais da pesquisa, seguiu-se uma seleção de palavras-chave que mais se alinhavam ao contexto de pesquisa. Para isso, utilizou-se a estratégia PICO, enriquecida com o elemento de Contexto (PICO+Contexto). O elemento "Comparação" (C) foi desconsiderado por ser opcional e não se aplicar ao escopo exploratório desta revisão, que busca mapear abordagens em vez de comparar diretamente grupos de controle. Assim, os parâmetros definidos foram:

População (P): programadores iniciantes e estudantes de ciência da computação (ex.: introductory programming, CS1, novice programmers);

Intervenção (I): o ensino e a aplicação de habilidades no processo de depuração de código (ex.: debugging skills, program repair);

Resultados (O - Outcomes): os impactos educacionais e comportamentais observados (ex.: learning outcomes, autonomy, academic performance);

Contexto (C): o uso de tecnologias ativas, especificamente a Inteligência Artificial Generativa e a Gamificação (ex.: Generative AI, LLM, game-based learning).

Com essa estruturação, elaborou-se uma string de busca genérica, agrupando os sinônimos de cada elemento com o operador lógico OR e interligando os grandes grupos com o operador AND, resultando na seguinte string:

("Introductory programming" OR "CS1" OR "novice programmers" OR "computer science students") AND ("Debugging Skills" OR "Debugging education" OR "Teaching debugging" OR "bug fixing" OR "program repair") AND ("Generative AI" OR "GenAI" OR "LLM" OR "Gamification" OR "game-based learning" OR "educational gamification") AND ("learning outcomes" OR "student perception" OR "autonomy" OR "instructional approaches" OR "academic performance")

A string supracitada foi adaptada para atender às especificidades sintáticas de três bases de dados científicas primárias, selecionadas por sua relevância e abrangência nas áreas de Ciência da Computação e Informática na Educação: ACM Digital Library, IEEE Xplore e Scopus. Para a base IEEE Xplore, especificamente, a sintaxe foi ajustada com o uso de caracteres curinga (asteriscos) para otimizar o alcance dos termos, o que resultou na seguinte string alternativa:

((("introductory programming" OR CS1 OR novice programm* OR "computer science student*") AND (debugg* OR "debugging education" OR "teaching debugging" OR "bug fix*" OR "program repair") AND ("generative AI" OR GenAI OR LLM OR gamif* OR "game-based learning" OR "educational gamification") AND ("learning outcome*" OR "student perception" OR autonom* OR "instructional approach*" OR "academic performance"))

No protocolo de pesquisa foi definido que a busca seria restrita aos trabalhos publicados entre os anos 2020 a 2025, para que fosse possibilitada a captura do estado da

arte mais recente, englobando as produções dentro do período de popularização massiva dos LLMs e seus impactos no cenário educacional.

C. Critérios de Seleção de Estudos

Foram definidos critérios rigorosos de inclusão e exclusão de artigos a fim de selecionar apenas estudos que contemplem as perguntas de pesquisa, mantendo a objetividade aos conteúdos com potencial de responder essas questões. Abaixo, são listados os critérios de Inclusão e de Exclusão utilizados na análise inicial dos artigos, na qual os trabalhos precisavam obrigatoriamente contemplar todos os critérios de Inclusão, além de não esbarrar em nenhum dos critérios de exclusão:

Inclusão: Estudos publicados entre 2020 e 2025; Artigos completos (full papers); Publicados em conferências ou periódicos revisados por pares; Estudos que abordam programadores iniciantes (ex.: CS1, introductory programming); Trabalhos que tratam explicitamente do ensino ou suporte ao debugging; Estudos que utilizam gamificação, Inteligência Artificial Generativa (GenAI/LLMs) ou ambos no contexto educacional; Artigos que apresentem avaliação empírica (experimentos, estudos de caso, surveys, etc.); Trabalhos que reportem resultados educacionais (ex.: aprendizagem, engajamento, autonomia, desempenho, percepção do aluno).

Exclusão: Estudos anteriores a 2020; Resumos, pôsteres, short papers ou artigos incompletos; Revisões sistemáticas, mapeamentos sistemáticos ou surveys (podem ser usados só como apoio teórico); Trabalhos que não abordam debugging diretamente; Estudos que não envolvem programadores iniciantes; Artigos que focam apenas em desenvolvimento técnico de ferramentas, sem avaliação educacional; Estudos que utilizam IA ou gamificação fora do contexto educacional; Artigos duplicados entre bases; Trabalhos sem acesso ao texto completo; Resumos de eventos.

A filtragem foi realizada a partir de um funil de seleção, em que se realizou a aplicação do filtro lendo os títulos e resumos dos artigos retornados com as pesquisas das respectivas bases de dados, classificando em aceitos, rejeitados ou duvidosos. Nessa fase não foram classificados artigos como duvidosos, dispensando a aplicação do filtro 2, que foi planejado como a leitura da introdução e conclusão para a decisão final. Esse processo de seleção e documentação segue as diretrizes atualizadas do *Preferred Reporting Items for Systematic reviews and*

Meta-Analyses (PRISMA 2020) [19], um protocolo concebido para assegurar a transparência, a completude e o rigor metodológico no relato das revisões sistemáticas.

Em aderência à recomendação do PRISMA de mapear o fluxo de triagem e detalhar as exclusões por motivos específicos [19], registra-se que a partir desse filtro inicial, ocorreu a exclusão de 91 trabalhos por serem resumos de eventos, 46 por não abordarem o debugging de forma direta e 32 por serem estudos secundários/RSLs, sendo alguns desses últimos aproveitados para o apoio teórico.

D. Avaliação de Qualidade e Extração de Dados

Após a filtragem inicial, os artigos selecionados foram lidos de forma integral e avaliados quanto a sua qualidade, garantindo que a robustez metodológica seja suficiente para a extração dos dados. Com isso, foram definidas 5 questões para avaliação, sendo elas:

- A metodologia do estudo é clara e bem descrita?
- A aplicação da gamificação/IAG foi claramente definida?
- O modelo ou proposta adotada é bem fundamentada?
- Há discussão e medição clara dos resultados educacionais alcançados?
- Os autores mencionam as ameaças à validade, desafios ou limitações do estudo?

Com essas questões definidas, realizaram-se as avaliações dos artigos com a definição dos pesos de acordo com o nível de atendimento, sendo 1,0 ponto para "Sim", 0,5 ponto para "Parcialmente" e 0,0 ponto para "Não". Estabeleceu-se como nota de corte a pontuação mínima de 2,5 pontos (50% do total máximo). Trabalhos com notas inferiores foram desclassificados. Ainda nessa fase, por conta da leitura integral, foram detectados critérios de exclusão que não foram detectados na filtragem inicial, resultando em novas remoções.

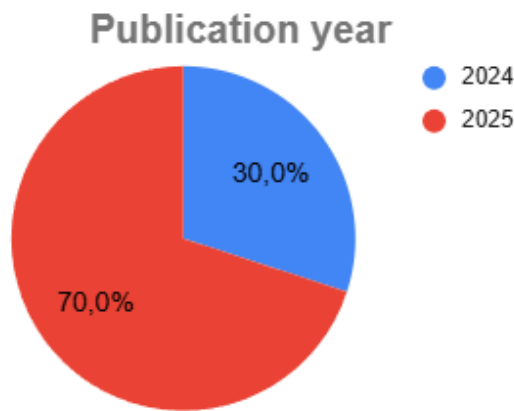
A partir da lista dos artigos aprovados na etapa de qualificação, uma nova leitura foi conduzida a fim de extrair estruturadamente os dados com o objetivo de coletar evidências e trechos para responder diretamente às questões de pesquisa (secundárias e de contexto), sendo as de contexto focadas em mapear: 1) o nível educacional (ex.: curso, graduação, escola), 2) conhecimentos prévios, 3) tamanho da turma amostral e 4) duração do experimento.

A partir dessas extrações, os trechos foram organizados em planilha para síntese e análise temática, que serviram para os relatos da seção de Resultados a seguir.

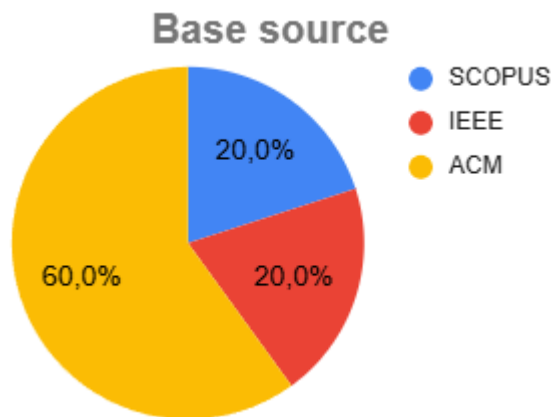
IV. Resultados

A. Metadados e Demografia das Publicações

A extração sistemática validou 10 estudos primários que abordam o processo de debugging mediado por inteligência artificial (IA) e gamificação, apresentando alto rigor metodológico (pontuação média de qualidade de 4,85)[20–29]. A distribuição temporal, ilustrada na Fig.1 (Ano de publicação), demonstra uma concentração absoluta nos últimos dois anos, com 70% das evidências datadas de 2025 e 30% de 2024.

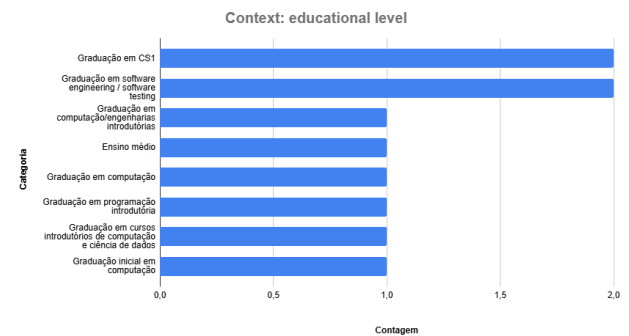


A Fig. 2 (fonte base) indica a predominância da biblioteca digital ACM (60%), seguida por IEEE Xplore e SCOPUS (20% cada). A análise geográfica consolidada na Fig.3 evidencia a liderança dos Estados Unidos (3 estudos)[20, 21, 22] e da Índia (2 estudos)[23, 24], com contribuições singulares da Alemanha[25], Brasil[26], Austrália[27], China[28] e Singapura[29].



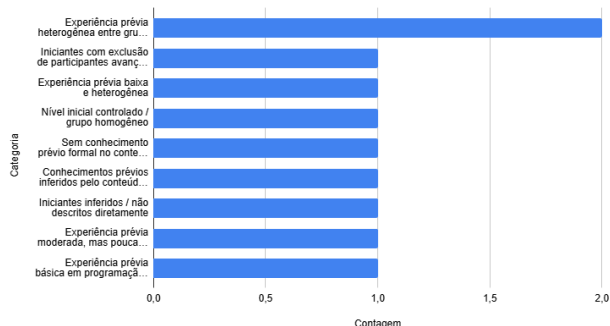
B. Contexto Educacional e Demográfico

Os dados populacionais extraídos atestam um foco direcionado ao ensino superior inicial. A Fig. 4 (Contexto: nível educacional) revela aplicações concentradas na graduação superior, com incidências primárias em disciplinas de "Graduação em CS1" (2 estudos) [21, 26] e "Graduação em software engineering/software testing" (2 estudos) [25, 29], além de disciplinas genéricas de programação introdutória [22, 28].

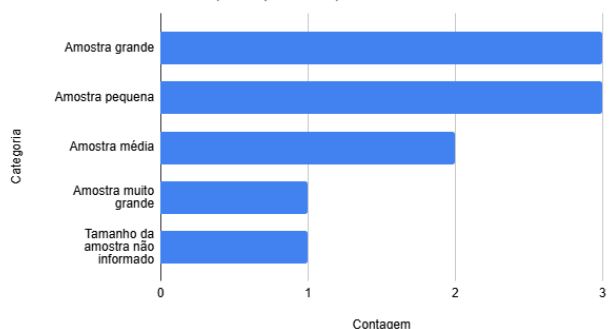


A Fig.5 (Contexto: Conhecimentos prévios) confirma o alinhamento das ferramentas com as necessidades de aprendizes inexperientes, categorizando as amostras como turmas com "Experiência prévia heterogênea entre grupos" (2 ocorrências) [26, 28] ou grupos de iniciantes com baixo conhecimento formal prévio [21, 22, 25]. Em relação ao delineamento metodológico, a Fig. 6 (Distribuição - Contexto (Tamanho amostra)) aponta um equilíbrio entre desenhos empíricos: amostras grandes (3 estudos) [22, 24, 28], amostras pequenas (3 estudos) [21, 25, 29] e médias (2 estudos) [23, 26]. A Fig. 7 (Distribuição - Contexto Duração)evidencia que as intervenções ocorreram sob monitoramento longitudinal, caracterizadas por múltiplas semanas de observação (3 estudos) [22, 24, 28] ou acompanhamento integral de um semestre letivo (3 estudos) [21, 26, 29].

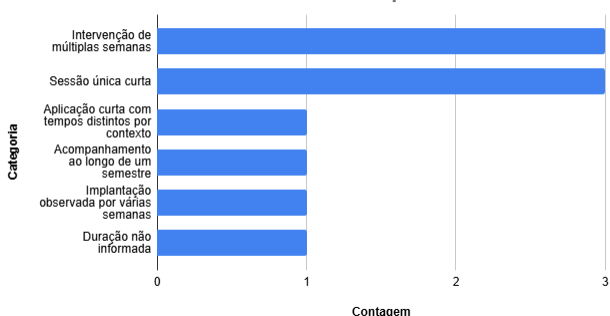
Distribution - Context (Prior knowledge)



Distribution - Context (Sample size)



Context: duration of the experiment

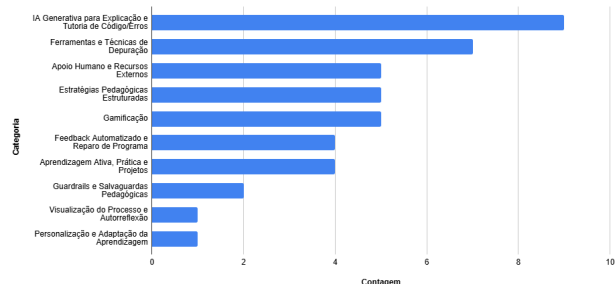


C. Tecnologias e Estratégias Pedagógicas (Q1)

A Fig. 8 (Categorias Q1) expõe as abordagens técnicas utilizadas para sanar a dificuldade discente com falhas de software. A categoria "IA Generativa para Explicação e Tutoria de Código/Erros" prevalece de forma isolada, conduzindo 9 dos 10 estudos primários [21-22, 24, 26-29].

Para neutralizar a automação passiva, as intervenções sintéticas foram atreladas a "Estratégias Pedagógicas Estruturadas" (6 ocorrências) [21, 24, 25, 26, 28, 29] e mecânicas de "Gamificação" (5 ocorrências) [21-25]. Adicionalmente, o gráfico aponta o uso de metodologias de "Aprendizagem Ativa, Prática e Projetos" em 4 trabalhos [20, 22-24], fundamentais para contextualizar o processo de depuração de falhas.

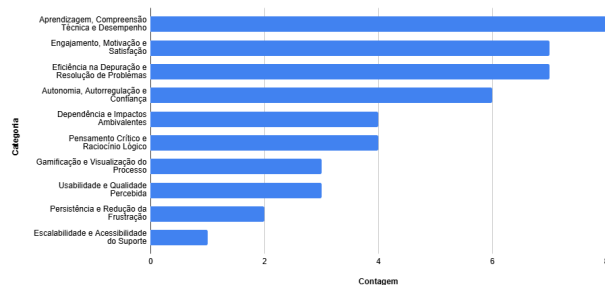
Q1 Categories



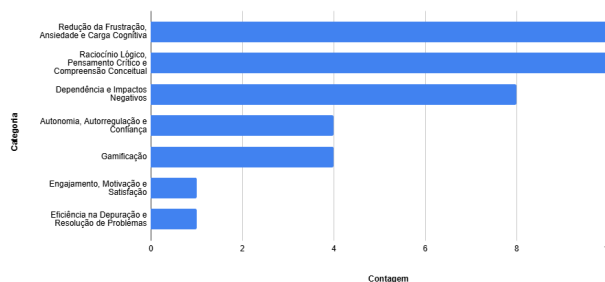
D. Benefícios Educacionais e Resolução de Problemas (QS2 e QS4)

A quantificação de desfechos positivos é detalhada nas Figs. 9 e 10 (Categorias QS2" e "Categorias QS4"). A IA atuou como mitigador de estresse, refletindo-se na "Redução da Frustração, Ansiedade e Carga Cognitiva", que registrou a marca máxima de 10 incidências empíricas [20-29]. O uso das ferramentas não limitou a capacidade investigativa, fomentando o "Raciocínio Lógico, Pensamento Crítico e Compreensão Conceitual" (10 ocorrências) [20-29].

QS2 Categories



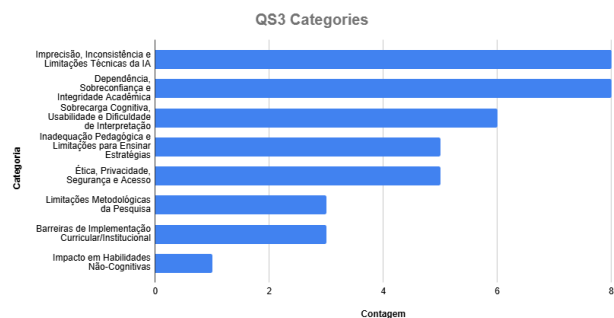
QS4 Categories



A Fig. 9 (QS2) reforça esses ganhos, indicando impactos diretos na "Aprendizagem, Compreensão Técnica e Desempenho" (8 menções). A sinergia entre explicações dialógicas e gamificação elevou a "Eficiência na Depuração e Resolução de Problemas" (7 ocorrências) e o "Engajamento, Motivação e Satisfação" (7 ocorrências). Como resultado comportamental, observou-se a consolidação da "Autonomia, Autorregulação e Confiança" em 6 avaliações.

E. Desafios, Limitações e Riscos (QS3)

A Fig. 11 (Categorias QS3) isola os obstáculos críticos inerentes à implementação da IA no processo educacional. A literatura relata uma severa "Dependência, Sobreconfiança e Integridade Acadêmica" (8 ocorrências), evidenciando o risco de os aprendizes utilizarem os agentes de IA como atalhos para a escrita automática do código.



Anomalias arquitetônicas manifestaram-se através da "Imprecisão, Inconsistência e Limitações Técnicas da IA" (8 menções), indicando a ocorrência de alucinações algorítmicas. Estudantes iniciantes também enfrentaram "Sobrecarga Cognitiva, Usabilidade e Dificuldade de Interpretação" (6 menções) ao operar comandos complexos (prompts). Questões de "Ética, Privacidade, Segurança e Acesso" foram identificadas em 6 estudos, acompanhadas por "Inadequação Pedagógica e Limitações para Ensinar Estratégias" (5 ocorrências), onde a ferramenta falhou em instruir a metodologia correta de debugging sem a intervenção humana.

V. Discussão

A. O Papel da IA Generativa no Ensino Introdutório

A demografia delineada nos gráficos de contexto (Nível educacional e Conhecimentos prévios) reafirma um diagnóstico histórico no ensino da computação: alunos de CS1 e engenharias enfrentam uma barreira de entrada severa devido à opacidade das mensagens de erro tradicionais de compiladores [21, 22, 26]. A prevalência massiva de "IA Generativa para Explicação e Tutoria" (9 ocorrências na Fig. Q1) demonstra uma ruptura metodológica [20, 28, 29]. Os tutores baseados em grandes modelos de linguagem (LLMs) assumem o papel de tradutores semânticos, convertendo falhas lógicas e de sintaxe em diálogos compreensíveis [30]. A forte ocorrência de "Estratégias Pedagógicas Estruturadas" e "Gamificação" na mesma Fig. (Q1) ressalta que o mero acesso à tecnologia é insuficiente; a eficácia da IA

depende de um delineamento instrucional que insira a tecnologia em lógicas de jogos sérios e aprendizagem baseada em projetos [23, 25, 31].

B. O Paradoxo do Alívio Cognitivo e a Dependência Algorítmica

O contraste entre os dados de impacto positivo (QS2 e QS4) e os riscos detectados (QS3) estabelece o principal paradoxo desta temática. Por um lado, as ferramentas sintéticas eliminaram o vetor primário de evasão universitária nestas disciplinas, traduzido nas 10 incidências de "Redução da Frustração e Carga Cognitiva" e 7 menções ao aumento do "Engajamento" [21-23, 30]. O ganho simultâneo em "Raciocínio Lógico e Pensamento Crítico" (10 ocorrências) corrobora a premissa de que a IA pode atuar como um facilitador cognitivo (scaffolding), que desobstrui impasses técnicos e permite que o aluno se concentre na lógica estrutural do algoritmo [31, 32].

Entretanto, as métricas da Fig. QS3 demonstram que esse alívio rapidamente se degenera em muleta tecnológica. A taxa empírica de "Dependência e Sobreconfiança" (8 ocorrências) indica que, desprovidos de atrito, os alunos inibem a capacidade investigativa e confiam cegamente na solução automatizada [20, 24, 26, 29, 33]. A IA repara o código, mas o processo de ensino de depuração (debugging skills) é terceirizado, anulando a autonomia futura do programador.

C. Implicações Pedagógicas e Limitações Técnicas

As "Inadequações Pedagógicas" (5 ocorrências) e as falhas por "Imprecisão e Limitações Técnicas" (8 ocorrências) reportadas na Fig. QS3 revelam que os LLMs atuais não possuem rigor de verificação epistemológica [6, 10, 15, 34]. A IA generativa prioriza a fluidez textual em detrimento da precisão técnica, resultando frequentemente em falsos positivos na localização de defeitos ou no encorajamento de reparações lógicas equivocadas [22, 27, 35].

Esta instabilidade técnica, aliada à "Sobrecarga Cognitiva e Dificuldade de Interpretação" (6 ocorrências), exige que instituições acadêmicas implementem protocolos de restrição (guardrails). A literatura evidencia que a sustentabilidade da inserção de IA em turmas de iniciantes requer a orquestração rigorosa do nível de ajuda fornecida pelo sistema [27, 32]. O fornecimento de resoluções completas (código-fonte pronto) deve ser sistemicamente bloqueado em favor de metodologias socráticas, onde o bot gamificado conduz o aluno a

identificar a própria falha por meio de questionamentos direcionados [25, 28, 32, 36].

VI. Ameaças à validade

Sobre ameaças à validade de construção, destaca-se o risco de a string não retornar todos os artigos relevantes existentes no mundo. Para minimizar esse risco, foi fundamental a utilização da estrutura PICO(C), a inclusão de uma ampla gama de sinônimos e a escolha de três das maiores bases de dados da computação (ACM, IEEE e Scopus).

Quanto à validade interna, reconhece-se que o impacto da ação humana pode enviesar a decisão de inclusão e exclusão dos estudos, bem como a nota de qualidade atribuída a eles. Para mitigar essa questão, padronizaram-se estritamente os critérios de elegibilidade e elaborou-se um questionário objetivo de qualidade, garantindo o alinhamento na forma de atuação da equipe durante a filtragem.

No que tange à validade externa, os resultados trazem o foco em turmas iniciantes de graduação e uso de LLMs específicos, assumindo-se que não são generalizáveis para alunos de níveis avançados ou para ferramentas de IAG ainda não testadas.

Por fim, quanto à validade de conclusão, como se trata de um tema extremamente recente, a pesquisa resultou em um número reduzido de artigos primários com alto rigor metodológico, sendo 70% deles datados em 2025, o que é esperado e suficiente para o presente mapeamento do estado da arte, necessitando de atualizações desta RSL devido à potencial elaboração de novos trabalhos dentro da temática nos anos futuros.

VII. Conclusão e Trabalhos Futuros

Esse estudo buscou mapear o estado da arte do ensino de debugging mediado por IAG e gamificação em turmas iniciantes de computação. Com as análises, constatou-se que o uso de IAG, como as LLMs, tem a capacidade de reduzir drasticamente a frustração inicial dos estudantes pela capacidade de respostas rápidas aos problemas que surjam na depuração. Porém, o uso inadequado dessas tecnologias como ferramentas irrestritas de facilitação, assumindo o papel de tradutores semânticos de falhas lógicas e de sintaxe sem um delineamento instrucional rigoroso, gera a potencialidade de criar a chamada “ilusão de competência”, inibindo o raciocínio crítico e prejudicando o aprendizado, podendo causar dependência severa.

Como descoberta central, evidencia-se que essa facilitação cognitiva traz consigo um paradoxo, onde de um lado proporciona alívio cognitivo (*scaffolding*), mas, do outro pode gerar uma dependência algorítmica. O *scaffolding* pode rapidamente se tornar uma “muleta tecnológica”, levando os aprendizes a perderem a percepção sobre a necessidade de investigar e refletir criticamente sobre os problemas para entender. Confiando irrestritamente nas soluções que essas ferramentas (sobreconfiança), terceirizam o processo de debugging para a IAG, tornando nula a possibilidade de autonomia futura como programadores profissionais.

Identifica-se também que as ferramentas de LLMs atuais priorizam a fluidez do texto, sem necessariamente trazer precisão sobre o que precisa ser aprendido e executado para a compreensão do processo, pois não possuem o necessário rigor de verificação epistemológica. Como resultado, as respostas podem indicar falsos positivos ou instruções de correção que sequer resolvem o problema, que podem inclusive gerar ainda mais frustração. Uma forma de contornar essa situação, a literatura aponta para implementação de protocolos de restrição (*guardrails*) nas plataformas educacionais, onde a principal recomendação é que exista um bloqueio no fornecimento de códigos-fonte prontos, restringindo as respostas a uma abordagem socrática e gamificada. Dessa maneira, garante-se que a interação seja por meio de perguntas e ações desafiadoras que conduzam o aluno à identificação autônoma e capacitação para resolução da falha.

Trabalhos futuros podem envolver o desenvolvimento prático de ferramentas gamificadas para utilização em turmas reais, contemplando uma tecnologia com um tutor socrático embarcado que utilize IAG, seguida da realização de testagens empíricas para avaliar o impacto e analisar a utilidade e eficácia dessa contextualização no aprendizado e na autonomia dos estudantes.

Declaração de Uso de IA

Durante a preparação deste trabalho, os autores utilizaram o assistente Google NotebookLM para auxiliar na extração estruturada de evidências dos artigos primários, na sumarização para análise temática e na revisão gramatical e de estilo do texto. Após o uso dessas ferramentas, os autores revisaram, editaram e validaram todo o conteúdo em estrita conformidade com o método científico, assumindo total responsabilidade pela integridade, originalidade e exatidão do conteúdo final desta publicação.

REFERENCES

- [1] U. Z. Ahmed, S. Sahai, B. Leong, and A. Karkare, "Feasibility Study of Augmenting Teaching Assistants with AI for CS1 Programming Feedback," in SIGCSE TS 2025.
- [2] D. J. Bouvier et al., "Teaching Programming Error Message Understanding," in CompEd 2023.
- [3] M. Venter, "Gamification in STEM programming courses: State of the art," in EDUCON 2020.
- [4] A. Almeida, D. D. S. Guerrero, and W. L. Andrade, "Fostering Problem-Solving in Introductory Programming Through Software Testing," in FIE 2025.
- [5] C. Kazimoglu, "Enhancing Confidence in Using Computational Thinking Skills via Playing a Serious Game," IEEE Access, 2020.
- [6] R. C. Sampaio, M. Sabbatini, and R. Limongi, "Diretrizes para o uso ético e responsável da Inteligência Artificial Generativa," Editora Intercom, 2024.
- [7] J. Prather et al., "The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers," in ICER '24.
- [8] S. Nguyen et al., "How Beginning Programmers and Code LLMs (Mis)read Each Other," in CHI '24.
- [9] D. J. Bouvier et al., "Teaching Programming Error Message Understanding," in CompEd-WGR 2023.
- [10] R. Batra and Z. Atiq, "Understanding Students' Frustration and Confusion during a Programming Task," in FIE 2023.
- [11] K. Banweer and D. Trytten, "A Qualitative Study of How Computer Science Students Develop Debugging Skills," in FIE 2024.
- [12] J. Penney et al., "Understanding Programming Students' Help-Seeking Preferences in the Era of Generative AI," in CompEd 2025.
- [13] X. Long et al., "Understanding and Enhancing CS Students Interaction Experience with AI Coding Assistant Tools," ACM Trans. Softw. Eng. Methodol., 2025.
- [14] R. Deoraj, J. V. S. G. Kurivella, and M. Moraes, "Guiding, Not Giving: Analyzing the Effectiveness of Guided Problem-Solving in CSI," in FIE 2025.
- [15] C. Ayora et al., "Rethinking How to Teach Analysis and Modelling of Business Requirements: A Serious Game Integrating GenAI," in MODELS-C 2025.
- [16] M. Triantafyllidou, H. Apostolidis, and L. Moussiades, "CODEWIZARDY, Online Application Supporting Instructors and Students to Learn Programming," in EDUCON 2024.
- [17] D. B. Silva, D. R. Carvalho, e C. N. Silla Jr., "A Clustering-Based Computational Model to Group Students With Similar Programming Skills," IEEE Transactions on Learning Technologies, 2024.
- [18] B. Kitchenham, "Procedures for performing systematic reviews," Keele University, Keele, UK, Joint Technical Report TR/SE-0401, 2004.
- [19] M. J. Page et al., "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," BMJ, vol. 372, n. 71, 2021. doi: 10.1136/bmj.n71
- [20] K. H. Levin et al., "ChatDBG: Augmenting Debugging with Large Language Models," Proc. ACM on Software Engineering (FSE), 2025.
- [21] S. Yang et al., "Debugging with an AI Tutor: Investigating Novice Help-seeking Behaviors and Perceived Learning," in ICER '24.
- [22] B. Adhikari, S. Dhakal, e A. Jha, "Maximizing Student Engagement in Coding Education with Explanatory AI," 2024.
- [23] S. V. P. Masetty e S. Vallamulla, "Enhancing Novice Programming Education through AI-Driven Mentorship and Project-Based Learning," VedaVerse, 2024.
- [24] M. Sivasakthi e A. Meenakshi, "Generative AI in Programming Education: Evaluating ChatGPT's Effect on Computational Thinking," SN Computer Science, 2025.
- [25] P. Straubinger, T. Greller, e G. Fraser, "Sojourner under Sabotage: A Serious Testing and Debugging Game," in FSE Companion '25, 2025.
- [26] L. N. Gomes, C. F. Fraga, M. Holanda, e D. Da Silva, "ChatGPT as a Teaching Assistant in a CS1 course: An Experience Report," 2024/2025.
- [27] Renzella et al., "Compiler-Integrated, Conversational AI for Debugging," 2025.
- [28] X. Gong, Z. Li, e A. Qiao, "Impact of generative AI dialogic feedback on different stages of programming problem solving," Education and Information Technologies, 2025.
- [29] O. Kurniawan et al., "Designing for Novice Debuggers: A Pilot Study on an AI-Assisted Debugging Tool," in Koli Calling '25, 2025.
- [30] W. Jang et al., "Analyzing Communication Logs in Pair Programming: A Comparison of Human- and LLM-Based Approaches," ETRA 2024.

- [31] H.-W. Chao e A. Y. Chang, "Exploring the Role of AI in Transforming Programming Education at Universities," 2025.
- [32] H. M. Jamil e X. Mou, "Automated Feedback and Authentic Assessment for Online Computational Thinking Tutoring Systems," in FIE 2017/2018.
- [33] X. Xiao et al., "An assessment framework of higher-order thinking skills based on fine-tuned large language models," Expert Systems With Applications, 2025.
- [34] Y. Yang, A. Mei, e C. Yang, "An Empirical Study of MBFL on Novice Programs Across Different Programming Languages," Int. Journal of Software Engineering and Knowledge Engineering, 2025.
- [35] Y. Zhou et al., "CodeEduBench: A Multidimensional Benchmark for Evaluating AI-Driven Code Generation Models," in ICAIES 2025.
- [36] S. Kannam et al., "Code Interviews: Design and Evaluation of a More Authentic Assessment for Introductory Programming Assignments," in SIGCSE 2025.